Final Project Introduction

Adam Meyers
New York University



Outline

- Goals
- Milestones
- Collaboration
- Types of Projects
- Examples
- Available Resources



General Goal: Write a paper that tells a story using 1 or more experiments

- The story should integrate:
  - The experiments and their results
    - The results should be measured according to a clear metric
    - The experiments should be valid
  - How do these experiments relate to previous work
    - Same/different metrics?
    - Same/different type of output?
  - A reason for doing the experiments, e.g.,
    - A human phenomenon is being modeled
    - A step towards solving some useful problem
    - A step towards implementing a useful program
- Some version of the experiment should demonstrate something that you can explain.



Goals (Restated)

- **A Well-written paper**
  - **Clear description including what worked and what didn't work**
- Use computer programs (and annotation, etc.) to conduct experiments and **integrate your results into the paper.**
  - **Multiple versions produce results**, even if low scoring
  - Comparison of versions provide insight into what does and doesn't work
  - Look at results on development data & identify why your system correctly classifies some things and misclassifies others
    - Examples – suppose POS tagger makes errors on allcaps – next slide
- **Relate to Previous Work**
  - Read/compare conference/journal/workshop papers to your work
  - Based on, better/worse scores, simpler system, different details, etc.
- **Impact Beyond this Class**
  - Step towards Conference submission
  - Distribute-able Software or Other Resource
    - Open source or Commercial
    - Need not be final version
  - Step towards further research



Example of error analysis on DEV – suppose POS tagger makes errors on allcaps

- cd ~/PENN-TREEBANK-TAG/
- grep -E '[A-Z]+\/' *tag | less
- /[A-Z]+\/
- Problem examples for allcaps → NNP
  - PC, TV, TV,OK
  - THREE COMPUTERS THAT CHANGED



Final Project Website

- https://cs.nyu.edu/courses/fall25/CSCI-UA.0469-001/final_project_info.html
- More detail than this talk
- Outline:
  - Schedule
  - Format
  - Guidelines
  - Examples
  - Downloadable Resources



Project Schedule/Milestones

- Discuss your ideas with a TA (or me) before and/or after filling out the questionnaire and/or writing the proposal.
- **Fill out Questionaire** – this includes partner requests, topics, abilities, etc.
- **I use the results to create final project teams** of 3–4 students and match each team to a graduate student mentor, who you should meet with on a weekly basis during the 2nd half of the semester.
- Write and submit a **final project proposal** with feedback from your mentor.
- **Weekly meetings** with your mentor (attendance will be taken)
- 3**0 second progress report:** Do 30 second informal talks about your project.
- **First Draft of Final Project:** late in semester – optional – only if you are ready.
- **Project Talks** during the last 3 classes of the semester
    - 6 minutes + 1 minute for questions
    - Guidelines to be provided
- **Final Project Report** due last day of class.



Create Team & Meet with Mentor

- Questionaire
  - Include preferred teammates and other information
  - Self-selection works best, but matches based on questionnaire can work.
  - Please make sure that your group is compatible with each other
  - All members should be “contact-able”
- The matching process results in groups of 3–4 students
  - Includes self-selection of preferred teammates
  - A TA mentor will be assigned as well
- Prepare proposal shortly after group is established
  - Consult Mentor
- Meet (via Zoom) with mentor on weekly basis until the end of semester.
  - Mentor will track progress and member participation (including attendance)
- Groups can meet additional times without the mentor if needed.



Proposal - Slide1

- **2 to 4** pages describing project and showing it is **plausible**.
- Problem Statement/Introduction (under ½ page **– do not make too long**).
- **Evaluation method – required**
  - Should include training, development & test data
    - **Cross validation provides ways of combining training & development**
    - **However, the test set should be kept separate.**
    - **You must avoid tuning your system to test data**
  - Should **include a score measuring the quality of output**
- Discuss parts of articles related to your topic
  - Only articles from NLP conference, journal or workshop
    - Search the database on: http://aclweb.org/anthology/
  - **Select at least 5 papers that were published in an NLP venue**
    - **Nonacademic and business sources do not count**
    - **arXiv papers do not count**
  - **How does each article relate to your proposed research?** Related or competing technique? evaluation method? task definition? etc.



Proposal - Slide2

- Strategy for Solving the Problem
  - **System Project:** Algorithm, plan for implementation; plan for evaluation; simple version that you can complete quickly and elaboration(s) that you want to implement if you have time. Possible baselines, e.g., suppose you always assign the most frequent tag?
  - **Resources**
    - **Annotation Project:** initial specifications, a small amount of annotation (e.g., 1 page), and a plan on how to achieve a modest amount of high-quality annotation.
    - **Lexicon Project**: initial specifications, a small number of lexical entries, a word list or method for deriving word list or dictionary (manual, automatic, semi-automatic)
  - **Evaluation Project**: what you plan to read, programs to download and test; evaluation techniques you plan to try; what problems with evaluation you might want to address. (Similar to system project).
- **Indicate the roles each member will play**
- Possible collaboration or competitions with other research groups e.g., shared datasets or compare methods?
- **You must have an evaluation plan – a way of showing that version N is better or worse than version N minus 1, N minus 2, ...**



The Final Project Proposal Complete by Class 13

- Counts as a Homework Assignment
  - https://cs.nyu.edu/courses/fall25/CSCI-UA.0469-001/homework6.html



First Draft

- **Optional opportunity for feedback before final due date**
  - Please do not submit if not ready for some review
  - Due class 25
- Tentative Introduction
- Combination of Outline and Paper
  - Include drafts of most sections of the final paper. A few unfinished sections could be represented by the section headings and a little text
- At least one experiment should have been conducted and the results should be described – use your evaluation metric (on development set)
  - Examples: baseline system tested, annotation quality results on sample text, evaluation results for 1 or 2 systems, etc.
- Previous Work: How results so far compare with previous work
- Next: Explanation (in outline form) of what you plan to do next
- **Evaluation of Results so far – a numeric score**



Format of Final Paper

- Projects include a **written paper of approximately 6-10 pages, not including references**. (**Longer papers discouraged**).
  - You may hand in other materials in addition to the paper
- **Do not submit a paper above 12 pages in length**
- Use ACL 2021 style guidelines (**suggested, not required**)
  - Written guidelines
    - https://acl-org.github.io/ACLPUB/formatting.html
  - Latex and Word templates
    - https://github.com/acl-org/ACLPUB/tree/master/templates
  - Overleaf
    - https://www.overleaf.com/read/crtcwgxzjskr
  - Some Advantages of Latex over Word
    - Automatically generate bibliography
    - Easier to represent some math formulas
    - Overleaf is a Web interface for Latex that compiles in quickly



Additional Types of Collaboration

- **Possible to collaborate with other teams, possibly with coordination by your mentor.**
- **A shared task with competing systems**
  - 1 group could create the task via annotation and specs
    - That group could also conduct the evaluation
    - They could create the shared task and advertise in the class (or via NYUClasses)
  - Several groups could make systems that run on same data
  - They could then also collaborate on an ensemble system, combining other systems
- **Trading tasks**, e.g., annotating for each other's tasks
- **Different Tasks that are part of a pipeline**, e.g.,
  - Student/Group 1 does NE extraction
  - Student/Group 2 does Relation extraction using those NEs
  - Student/Group 3 does coreference using NEs
    - relations extracted by Group 2 important feature for coreference system
- **Collaboration requires definitions and samples of input and output**
  - Use existing data while collaborators work on their part of pipeline



Considerations for Group Projects

- Divide labor based on individual strengths
- Code writing
  - Version control
  - Write separate components & agree on input and output specifications
  - If different programming languages, consider using file I/O
  - **Coordination between programmers requires planning**
- Possibly choose different parts of a pipeline
  - $\text{input} \rightarrow \text{system}_1 \rightarrow \text{system}_2 \rightarrow \ldots \rightarrow \text{system}_{N-1} \rightarrow \text{output}$
  - Can run on either “gold” or “system” data



Group Politics

- Be explicit about who is responsible for what
- Assign separate tasks – don't just say we will all do X together
- Change responsibilities if it turns out that one person has too much work
- Do not have one person write all the code, unless there are other components that are similarly labor intensive
- Similar considerations if your group needs to collaborate with other groups
- Beware of group members who are difficult to "reach" or who do not contribute on a regular basis
  - Weekly meetings, even 15 minute meetings are crucial
  - Each person should have something to report every week
  - Mentor will take attendance
- Make sure everyone has the same basic understanding of your project



Types of Final Projects

- **System Project** – a working program that performs a clearly defined task, evaluation and a write-up.
  - Includes original NLP implementation
  - Can use output of other systems, but should not be only combo of other systems
- **Resource** (lexicon, annotated corpus, etc.) – specifications, evaluation and write-up.
- **Evaluation Project** – Read papers about a specific area of NLP; download and test systems from this area; Implement existing methods for evaluation and possibly your own method. Compare evaluation methods: how they work, what they model, differences between systems given different measures, etc. Argue for some point of view.



Sample System, Annotation and Evaluation Papers

- https://cs.nyu.edu/courses/fall25/CSCI-UA.0469-001/final_project_info.html#Example%20Papers
- Examples of each paper type
- These can be used as models for how to structure your arguments
- 3 papers that are “Annotated as a model for paper writing”
  - Sections are labeled for overall function in the paper
  - However, any sample paper can be used as a model
  - In fact, the papers you cite can be models for style as well.



Sample Projects

- Some sample final project descriptions are provided here:
  - https://cs.nyu.edu/courses/fall25/CSCI-UA.0469-001/final_project_info.html#Topics
  - I may update this periodically.
  - Do not expect these project ideas to be complete:
    - Some of these topics are hypothetical and may need to be modified to obtain good results.
    - Other topics are somewhat general and you will need to work on specifics to make a project work.



No Sentiment Analysis Please!

- Sentiment Analysis has been a very popular topic over the years – in fact, too popular.
- **Most common type =** use reviews to predict positive/negative sentiments or numbers of stars
- Some problems:
  - Too much focus on ML problem (ignoring properties of data)
  - Handling negation words like “no” and “never”
  - Bad quality annotation
  - Insufficient error analysis
  - Very few novel contributions
  - Too easy to do a bad version quickly



No Prompt Engineering Projects

- Prompt Engineering
  - Experiments to optimize text prompts to LLMs
  - Can be achieved with little or no programming
  - Results may be “non-transparent”
- What you can do with LLMs and prompts
  - Use LLM + prompt for upper bound, lower bound or other baselines to show how well (versions of) your system work
  - It may even be difficult to beat these baselines, but it may be possible to figure out why



Other Guidance

- Topic should be related to topics covered in class.
- Allow for some transparency of the results
  - Do error analysis of the dev and/or training set – seek to understand errors based on data properties
- If you tune parameters to get a result, the whole paper should not be about the tuning process.



Example of How to Work Together

- Student$_1$ and Student$_2$ invent a new task: automatic classification of recipes
  - Recipe = appetizer, dessert; main course; lunch; dinner; breakfast;
  - Ingredients = main ingredient; minor ingredient; garnish; other
- Student$_1$:
  - Writes annotation guidelines
  - Writes up an entry program
  - Annotates data 1 time
  - Adjudicates multiply annotated data
  - Choosing some features for final ML system
  - Designs and runs evaluation on system
- Student$_2$:
  - Sets up corpus: acquires corpus, formats corpus, divides into train, dev and test
  - Writes baseline system for task
  - Annotates data 1 time
  - Coordinates tests of algorithms in Machine Learning Package
  - Chooses some features for final system



How to Get Started

- Find an area that interests you and read at least 5 academic papers in that area (to discuss in the proposal).
- A possible goal is to produce a paper similar to one of the ones that you read
- I don't expect a conference-grade paper in ½ semester of work
  - Correct experimental procedure is more important than stellar results
- **Figure out how you can evaluate your results**
  - How do they evaluate systems in related work?
  - Be prepared to do “error analysis” on results on development corpus
- You should read a **moderate** amount:
  - References can lead to more references
  - Reading shouldn’t get in the way of implementing code or writing paper
  - Don't code and write in a vacuum (look at papers)



How to Get Ideas

- Look at Examples from Final Paper Website
- Is there a homework assignment that you would like to “extend” in some way?
- Start with a simple idea and extend it
- Work with your mentor
- Search the ACL Anthology based on initial ideas and find papers that you find interesting.
- Come talk to me



Guidelines

- Base style on published conference papers and online guidelines
- Structure and Schedule your research
- Meet regularly with your mentor
- Multiple Versions: baselines and at least 1 system that “works”
- Incorporate previous work, but Implement at least 1 component yourself
- Compare to previous work (read before and while you implement)
- Split Data: training, development & test
- Group projects: document and plan collaboration
- Do not focus on parameter setting mechanisms – I don’t care which hyper-parameters of BURT you adjusted (etc.)
- Explain or describe patterns of error (see development corpus results)
- No papers about LLM prompt writing please (unless you are just using this as a baseline you try to beat using more interesting methods)
- No Sentiment Analysis papers
- More details here:
  - https://cs.nyu.edu/courses/fall25/CSCI-UA.0469-001/final_project_info.html#Guidelines



Baseline and Full Systems for Programming Projects

- It is useful to compare your results to **baselines**, systems used for purposes of comparison (both as lower and upper bounds)
- **Low baseline systems** can represent an obvious methods that any system should beat. **Upper bounds** can represent human performance or a "large" model based on lot more training data than you could replicate (e.g. LLM).
  - For example, for POS taggers, choose most frequent tag for each word based on training corpus, and choose NN for all unknown words.
    - It is possible that such a system could get 75% accuracy (suppose 50% of tokens have only 1 possible tag and another 25% have one tag that makes up at least 50% of the instances).
- Higher **baseline systems** may be proof-of-concept systems, very basic implementations of what you are doing before adding bells & whistles.
- **It is a good idea to mention one or more baselines in your proposal. It is a good idea to guarantee that you can produce something simple, even if you have more exciting things you would like to try.**



Annotation Projects

- Write Specifications & Annotate Documents
  - Find related work and distinguish your approach
  - Find one annotator in addition to yourself, so it is possible to evaluate
    - InterAnnotator Agreement

$$Kappa = \frac{Percent(Actual\ Agreement) - Prob(Chance\ Agreement)}{1 - Prob(Chance\ Agreement)}$$

    - Score (precision, recall, f-measure) against a gold standard
      - Multiply annotate and adjudicate to create sample
  - Possible deals:
    - Agree to annotate for another project in exchange for them annotating for you
    - Programmer uses annotation for program project
- Design and Implement Annotation Project using Amazon Turk
  - Figure out a way to use very simple annotation
  - Design a task for Amazon Turk and Run it
  - May cost you a little money ($50 buys a lot of annotation)



Baseline for Annotation Projects

- As an initial experiment, annotate a simpler version of your task to see if you can get reasonable agreement.
- Try Collapsing distinctions in your annotation for purposes of evaluation
  - Example: 2 sense of the adverb ***logically***
    - ***Logically, they should be able to do it.*** [VIEWPOINT]
    - ***They should be able to solve the problem logically*** [MANNER]
  - If collapsing these sense for evaluation purposes, gives you a better agreement score, you might report both results.
- Try Only annotating a subclass of your data or part of your specs
  - Example1: You are annotating instances of attacks reported in blogs, along with arguments of the attacks (attacker and victim)
    - Initially only annotate sentences containing a limited number of attack words (e.g., ***attack, harm, kill***, ...)
  - Example2: You are annotating attack and movement events in news
    - Initially, only annotate ***attack*** events



Crowd Sourced Annotation: Getting Good Results

- Detailed Specifications (true for any annotation)
- Simplify Task (not expert annotators)
- Use a qualification test (keep out trolls)
- Do short tests of your task
  - Fix task until you are satisfied with results
  - Then run on larger amount of data
- Multiply annotate data and merge, e.g.,
  - 30 groups of 25 examples = 750
  - Each group of 25 is annotated 3 times = 2250
  - 10 annotators each annotate 225 examples (+ 25 qualifying examples)
  - Sample merging strategy: voting
    - If binary classification, always can pick one answer
- See: http://crowdsourcing-class.org/tutorial_slides.html



Consider Turning Your Paper into a Conference Paper

- It is Possible that a Really Good Paper could form the basis of a Conference Paper
- If applicable, I will help you find a good venue: conference or workshop
- A published paper can be a major factor in getting into a good graduate program, particularly a PhD program
- Note that most publications have multiple authors, i.e., collaboration may help



Please Ask

- If you need help finding resources or citations relevant to your project, please send me email.

